

Évaluation Écrite – Module DevOps (CI/CD/CM)

1. Compléter le code Jenkinsfile

```
pipeline {  
    agent any  
  
    stages {  
        stage('Checkout') {  
            steps {  
                git branch: 'main', url: 'https://github.com/utilisateur/repo.git'  
            }  
        }  
  
        stage('Build') {  
            steps {  
                sh 'mvn compile -DskipTests'  
            }  
        }  
  
        stage('SonarQube Analysis') {  
            steps {  
                withSonarQubeEnv('sonarqube-server') {  
                    sh 'mvn sonar:sonar'  
                }  
            }  
        }  
    }  
}
```

Rôle du bloc withSonarQubeEnv :

Ce bloc configure automatiquement les paramètres de connexion au serveur SonarQube (URL, token d'authentification) et injecte les variables d'environnement nécessaires pour l'analyse SonarQube.

2. Explication du workflow CI/CD

Cheminement après un push sur Git :

- **Jenkins** détecte automatiquement le changement via un webhook et déclenche l'exécution de la pipeline
- **Maven** compile le code source et gère les dépendances
- **SonarQube** analyse la qualité du code (bugs, vulnérabilités, code smells, couverture de tests)
- **Sortie attendue** : Un artefact compilé et un rapport de qualité de code accessible via l'interface SonarQube

3. Étapes de déploiement Docker

1

Construire l'image

```
docker build -t mon-utilisateur/mon-app-springboot:latest
```

2

Exécuter localement

```
docker run -d -p 8080:8080 --name mon-app mon-utilisateur/mon-app-springboot:latest
```

3

Pousser sur Docker Hub

```
docker login
```

```
docker push mon-utilisateur/mon-app-springboot:latest
```

4. Corriger le Dockerfile

```
FROM openjdk:17-jdk-slim
```

```
COPY target/app.jar app.jar
```

```
CMD ["java", "-jar", "app.jar"]
```

Pourquoi une image JDK officielle :

- Plus légère et optimisée pour Java
- Sécurisée et maintenue régulièrement
- Évite l'installation manuelle du JDK
- Meilleures pratiques Docker

5. Compléter le fichier YAML

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: mysql
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: mysql
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: mysql
```

```
    spec:
```

```
      containers:
```

```
        - name: mysql
```

```
          image: mysql:8.0
```

```
          env:
```

```
            - name: MYSQL_ROOT_PASSWORD
```

```
              value: mot-de-passe
```

```
          ports:
```

```
            - containerPort: 3306
```

Commande d'application :

```
kubectl apply -f mysql-deployment.yaml
```

6. Workflow Kubernetes

Après création du Deployment :

- **Composants intervenants** : kube-apiserver, kube-controller-manager, kube-scheduler, kubelet
- **Maintien du desired state** : Le contrôleur de déploiement surveille en permanence l'état actuel et le compare avec l'état désiré
- **En cas de panne d'un pod** : Le ReplicaSet détecte la différence et crée un nouveau pod pour maintenir le nombre de réplicas souhaité

7. Rôle de Prometheus & Grafana

Surveillance dans Kubernetes :

- **Prometheus** collecte les métriques (CPU, mémoire, réseau) des pods et nodes
- **Grafana** visualise les données via des dashboards personnalisables

Détection dans CI/CD :

- Surveillance des métriques de build (durée, succès/échec)
- Alertes sur les dégradations de performance
- Détection des anomalies dans les tests

8. Workflow global CI/CD/CM

Rôle de chaque outil :

- **Git** : Gestion du code source et collaboration
- **Jenkins** : Orchestration de la pipeline d'intégration continue
- **Maven** : Build, gestion des dépendances et packaging
- **SonarQube** : Analyse qualité du code et sécurité
- **Docker** : Containerisation de l'application
- **Kubernetes** : Orchestration des conteneurs en production
- **Prometheus/Grafana** : Monitoring et observabilité

Interaction : Chaque outil transmet des artefacts ou métriques au suivant, créant un flux automatisé du commit jusqu'au déploiement et monitoring en production, avec retour d'information constant pour amélioration continue.

notes:

sonarcube:

metric.....

docker:

différance entre CMD et RUN

cmd se fait dans exécution du conteuneur

run se fait dans le build de l image